2025 International Conference on
Unmanned Aircraft Systems (ICUAS)
May 14-17, 2025. North Carolina, USA

Visual Control and Mapping for UAV-based Platform Inspection

Leonardo A. Fagundes-Junior[1], Carlos M. Soria[2], Raquel F. Vassallo[3], and Alexandre S. Brandão[1]

Abstract— **Autonomous take-off and landing capabilities are crucial in UAV vision-based missions, ensuring adaptive navigation, especially in challenging environments where real-time identification and interaction with a variety of landing platforms are required. In this context, this paper presents a servo-visual controller that uses pattern detection and color segmentation techniques to identify take-off/landing platforms and estimate their current orientation. The proposed system was subjected to experimental validation with four platforms positioned in different orientations, heights, and positions, demonstrating its versatility in various conditions. Our study addresses the Flying Robots Trial League challenge, which emulates mapping and inspection tasks in offshore platforms.**

I. Introduction

Unmanned aerial vehicles (UAVs) have emerged as indispensable tools in various fields, including autonomously or remote control inspection tasks [1]. Their ability to access remote or dangerous environments autonomously has revolutionized the execution of such operations. Indeed, UAVs provide fast and cost-effective solutions for infrastructure assessment, including bridge inspection, forest fire detection, pollution monitoring, pipeline surveillance, and building inspection, providing high-resolution optical or thermal images and data for analysis [2]–[5]. In addition, drones play a key role in land mapping and environmental monitoring, providing detailed spatial data for a variety of applications, from urban planning to ecological conservation. In essence, the versatility and efficiency of drones make them indispensable in a spectrum of fields that rely on remote operations.

Several studies, including those cited in [6]–[8], have demonstrated UAV applications encompassing intricate missions, such as autonomously collecting/handling/delivering packages to/from unmanned ground vehicles (UGVs), inspecting ground vehicles, and landing UAVs atop unmanned surface vehicles (USVs) in challenging water conditions for cleanup missions. These missions inherently require considerations for designated take-off and landing points within the operational scenario. Spatially, the UAV's trajectory must initiate and conclude at specific anchor points within the environment. These points can either be static or mobile, with their states known or unknown to the UAV. Notably, the task complexity escalates when the take-off/landing site exhibits greater degrees of freedom in movement, necessitating the estimation or prediction of its posture and, in some instances, its velocity. Temporally, the UAV must determine the optimal initiation time for the landing/take-off procedure, particularly when the mission involves a mobile platform.

Landing and take-off strategies on UAV platforms vary according to mission conditions. In scenarios where the landing base is static and its position is known to the UAV, the main challenge is to ensure precise landings. Barcelos et al. [6] present a cargo transportation strategy in which UAVs collaborate with unmanned ground vehicles (UGVs), aided by a motion capture system (MoCap). When the landing pad location is fixed but unknown to the UAV, on-board sensory data is crucial for accuracy. Khazetdinov et al. [9] uses an ArUco fiducial marker, with an additional internal marker placed on the landing surface for continuous visibility within the camera angle. Lebedev et al. [10] combine algorithms for precise landing based on the detection of the ArUco marker, capable of obtaining a 2-5 centimeter error between the robot's final position and the center of the landing platform. For their part, Sales et al. [11] identify landing platforms through computer vision and optimize visitation using K-nearest neighbor (KNN) for solutions to the traveling salesman problem (TSP). For mobile platforms with known posture, estimating and predicting motion are critical for safe landings. Chang et al. [12] propose an autonomous system for UAVs to land on moving platforms, such as automobiles or marine vessels, offering advantages for long-endurance flights. Grounded mobile platforms use MoCap for ground truth information and can estimate UAV posture by detecting an ArUco marker coupled to the UAV's body frame. Rabelo et al. [13] employ a line-shaped virtual structure controller that enables cooperative navigation between the UAV and UGV. In this approach, the virtual structure is dynamically adjusted to guide the UAV to hover directly above the UGV. By gradually decreasing the distance between them, the UAV lands safely on the UGV. Finally, in scenarios where both the platform and its posture are mobile and unknown, advanced control strategies are required for autonomous landing. In [14], a visual servoing controller generates velocity commands directly in image space, eliminating the need for a separate altitude controller. The work presents a strategy to search a circle on the image based on the landing zone pattern utilized at the MBZIRC competition. Similarly, in [8], a model predictive control (MPC) approach enables a UAV to land autonomously on a USV in harsh conditions. This method leverages visual data from an onboard camera to predict the vessel's nonlinear motion, estimating its pose when encountering an AprilTag marker affixed to it.

*This work was supported by FAPEMIG and CNPq.

[1]Núcleo de Especialização em Robótica, Department of Electrical Engineering, and Graduate Program on Computer Science, Federal University of Viçosa, Viçosa - MG, 36570-009, Brazil, E-mail: {leonardo.fagundes, alexandre.brandao}@ufv.br

[2]Instituto de Automática, at Facultad de Ingeniería, Universidad Nacional de San Juan, Argentine, E-mail: csoria@inaut.unsj.edu.ar

[3]Department of Electric Engineering, Federal University of Espirito Santo, Vitória - ES, Brazil, E-mail: raquel.vassallo@ufes.br

2025 International Conference on Unmanned Aircraft Systems (ICUAS) | 979-8-3315-1328-3/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICUAS65942.2025.11007867

In general, the literature converges on the use of computer vision-based techniques for platform identification, with various types of visual markers being employed for orientation, including ArUco markers, QR codes, AprilTag, and customized markers. These techniques facilitate precise take-off and landing operations, which are essential for successful UAV missions in a variety of environments. Nevertheless, specific scenarios present challenges when predefined search patterns are not available, as demonstrated in the aforementioned studies. In such cases, the identification of intrinsic environmental characteristics, such as distinct color schemes, becomes essential. In this context, the Flying Robot Trial League (FRTL), a category of the Latin American Robotics Competition (LARC), offers a platform for teams to showcase innovations involving aerial robots.

To better explain, the paper is split into the following sections. Section II presents the FRTL challenge and describes the main arena localization/mapping strategy, detailing the conversion of image plane features into 3D world coordinates. Section III presents the approach to solving the challenge using the proposed methodology, outlining strategies for mapping the platforms within the environment. Finally, Section IV presents the results of real experiments, while Section V summarizes the main conclusions of the study and elucidates possible avenues for future research.

II. The Proposed Vision-Based Navigation

This work proposes a strategy to identify landing/take-off platforms for UAVs and control their landing using servo-visual control in three-dimensional space. The platforms under consideration are planar bases positioned on the ground or at a fixed height, either stationary or in motion. This information is unknown to the UAV, as well as the posture of each base in the environment. Dimensional and color features are utilized to segment regions in the image plane, allowing for the localization, extraction, and estimation of platform information such as area and orientation. The robot navigates until it locates the platform and then employs positioning or trajectory tracking servo-visual control in the world frame. This involves transforming the features identified in the visual image captured by the UAV's camera into an estimate of their spatial representation in Cartesian coordinates.

To tackle the challenges presented at FRTL, we used the Parrot Bebop 2 drone equipped with its camera facing downwards. This configuration allowed the UAV to observe the platforms designated for landing and take-off. With this visual input, the control system was able to efficiently assess the approach and docking on the platforms. It is important to clarify that although information about the UAV's posture is essential for navigation during mission execution, it is beyond the scope of our current work. Our focus here is on wandering around the scene in order to locate and subsequently dock at the platforms. The UAV's posture information is derived from odometry, integrating on-board sensor data, but when it is close to a platform, its global position becomes relative to the platform itself.

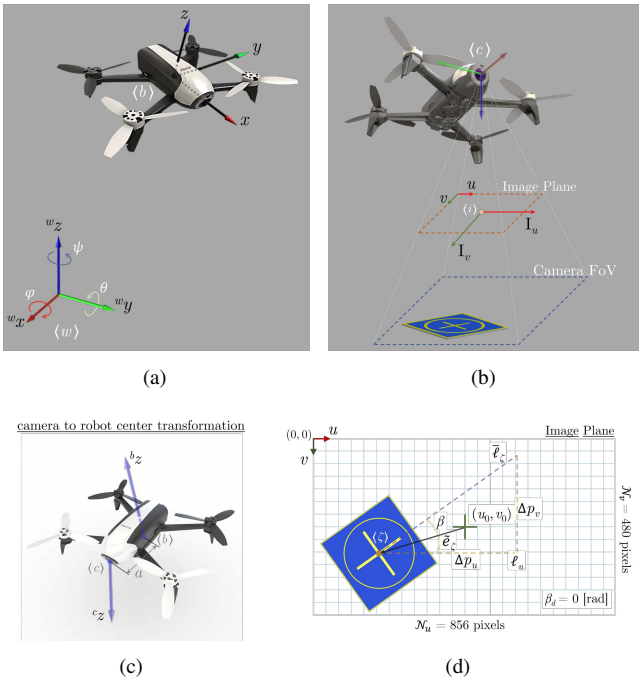

Fig. 1. Reference coordinate frames of the implemented system and strategy proposed to extract features from the robot's image and use it on the visual servoing control. Also, the definition of the posture error measurement in the image plane. (a) Parrot Bebop 2 description and pose variables, according to Tait-Bryan angles notation. (b) UAV's camera representation. (c) Relation between the camera and the robot's center. (d) Image projection on the 2D plan and land base segmentation during streaming.

A. The Adopted Notation

In this work, we establish the following definitions: a global frame denoted as $\mathcal{W} = \{\langle w\rangle, {}^{w}x, {}^{w}y, {}^{w}z\}$, and the UAV's reference frame denoted by $\mathcal{B} = \{\langle b\rangle, {}^{b}x, {}^{b}y, {}^{b}z\}$ located at the UAV's center of mass (CoM), as illustrated in Figure 1(a). Additionally, the camera frame can be defined as $\mathcal{C} = \{\langle c\rangle, {}^{c}x, {}^{c}y, {}^{c}z\}$, where $\langle c\rangle$ aligns with the optical center of the camera and with the 2D digital image plane, represented by the principal point $\mathcal{I} = \{\langle i\rangle, I_u, I_v\}$, as depicted in Figure 1(b). Finally, the estimated center point and orientation of the base in the image plane is denoted as $\mathcal{E} = \{\langle\zeta\rangle, {}^{i}x, {}^{i}y, {}^{i}\beta\}$.

Furthermore, we use the symbol $\mathbf{R}_{\mathcal{C}}^{\mathcal{B}}$ to represent a rotational matrix from the camera frame $\mathcal{C}$ to the UAV's frame $\mathcal{B}$. Vectors are represented by bold quantities as follows: ${}^{w}\mathbf{q} \in \mathbb{R}^n$, utilizing the frame $\mathcal{W}$ as a prefix; matrices are indicated in bold capital letters, such as $\mathbf{Q} \in \mathbb{R}^{n\times m}$.

B. Image Features Extraction

The process of pattern identification in the scene involves processing images obtained from the Bebop camera stream. This methodology relies on segmenting specific colors and detecting regions with predetermined characteristics, including shape (such as circular, square, or cross), centroid, vertex position, and bounding box. The conversion from RGB to HSV color space is essential for color segmentation, with predefined limits set for blue and yellow colors. Calibration

establishes boundaries for each segmented color, while a minimum area criterion reduces noise and false positives. Our image processing system beginning with the calibration of the robot camera's intrinsic parameters, followed by RGB frame capture and conversion to HSV, and concluding with color segmentation. The algorithm iterates through image digitization, binarization, noise filtering, and region extraction, selecting regions above a specified area threshold. The identified regions are then matched with known features to isolate potential landing pads, with a particular focus on identifying blue and yellow colors.

C. From Image Plane to Cartesian Coordinates

As show in Figures 1(b) and 1(d), exist a relative distance a between the camera $\langle c \rangle$ and the robot CoG $\langle b \rangle$ reference frames. This displacement is taken into consideration in the control proposal, which is performed in the inertial coordinates frame $\langle w \rangle$. The primary objective is to guide the robot to the center of the bases by controlling its horizontal position and vertical quote in the workspace, with cartesian coordinates, ensuring a smooth and continuous landing.

In the image plane, the control point is in the geometric center of the image as ${}^{i}\mathbf{x}_d = \begin{bmatrix} u_0 & v_0 \end{bmatrix}^\top$, in $\langle i \rangle$, depicted by the green cross in Figure 1(d), while the estimated centroid of the base is denoted as $\bar{\boldsymbol{\zeta}} = \begin{bmatrix} {}^{i}u & {}^{i}v \end{bmatrix}^\top \in \mathbb{R}^2$, with orientation ${}^{i}\beta$, represented by the circle asterisk, in pixel coordinates. For control purposes, the position error in the image plane is defined as $\tilde{\mathbf{e}}_\zeta = {}^{i}\mathbf{x}_d - \bar{\boldsymbol{\zeta}}$, as illustrated in Figure 1(d).

Estimating the current posture of the platform in Cartesian space aims to land the UAV parallel to two edges of the yellow cross and perpendicular to the remaining two edges, as shown in Figure 1(d). This strategy optimizes the UAV's landing area, as diagonally oriented landings on square platforms are more susceptible to failures, especially near the platform's vertices. By comparing the centroids of the largest blue area with the centroid of the smallest yellow region, delimited by the smallest convex region within the blue area, the position of the yellow cross can be precisely determined. Subsequently, the end of the cross closest to the I_u horizontal axis is identified and taken as a reference for orientation. This end is represented by the dashed line projected in purple, denoted as $\bar{\ell}_\zeta$, on the image plane. After estimating the position of the centroid of the cross and one of its extremities, the algorithm determines its orientation based on the Euclidean distance between these two points. By considering the pixel differences Δp_x and Δp_y on the projected line ℓ_u in the u and v directions, respectively, the estimated angle of the platform is calculated as ${}^{i}\beta = \arctan\left(\Delta p_y / \Delta p_x\right)$. Figure 2 illustrates the result of a more accurate estimate of the platform's orientation.

D. Visual Servoing Controller

In our previous work [15], a servo-visual control runs a PID compensator in the image plane to regulate UAV landing on a base. However, this approach had limitations, including sensitivity to uncertainties and the need to switch between control strategies in different spatial dimensions, potentially leading to instability. To address these issues, this present study proposes a new servo-visual control method based on cartesian space rather than the image plane. This approach involves transforming the reference from the image plane to three-dimensional space relative to the camera reference, allowing for more stable control and improved performance. The transformation is achieved through a homogeneous transformation that considers both the object's identification in the camera's reference frame and the drone's posture. By utilizing intrinsic and extrinsic camera parameters, points in the image plane can be accurately represented in the metric system, facilitating their conversion to world coordinates.

(a) UAV's camera original frame. (b) Feature extraction and segment.

Fig. 2. The posture error measurement in the image plane and the estimated orientation of the landing base. In this scenario, the computed orientation is ${}^{i}\beta$ = 7.64°, while the actual one is ${}^{w}\beta$ = 7.97°. The filled rectangle, depicted in transparent purple, represents the identified yellow cross, while cyan markers indicate the centroid calculation of the cross and its orientation. The red square marker represents the calculation of the estimated centroid of the blue regions corresponding to the identified base.

Therefore, with the control objective of taking the estimated point, represented in $\langle i \rangle$, to the principal point of the image, the following transformation arises from a point in the image $\bar{\boldsymbol{\zeta}}$ to a desired point in the three-dimensional space, in the inertial reference frame, where $\mathbf{x}_b$ is the current pose of the drone. Thus, for ${}^{c}\mathbf{x}_\zeta = \begin{bmatrix} {}^{c}x_\zeta & {}^{c}y_\zeta & {}^{c}z_\zeta & {}^{c}\beta \end{bmatrix}^\top \in \mathbb{R}^4$ being the image features in $\langle c \rangle$, the platform's position ${}^{w}\mathbf{x}_\zeta = \begin{bmatrix} {}^{w}x_\zeta & {}^{w}y_\zeta & {}^{w}z_\zeta & {}^{w}\beta \end{bmatrix}^\top \in \mathbb{R}^4$ can be obtained by

$$ {}^{w}\mathbf{x}_\zeta = \mathbf{R}^{\mathcal{W}}_{\mathcal{B}} \left(\mathbf{R}^{\mathcal{B}}_{\mathcal{C}} \, {}^{c}\mathbf{x}_\zeta + \begin{bmatrix} a & \mathbf{0}_{1\times 3} \end{bmatrix}^\top \right) + \mathbf{x}_b, \tag{1} $$

with $\mathbf{R}^{\mathcal{W}}_{\mathcal{B}} = \begin{bmatrix} c_\psi & -s_\psi & 0 & 0 \\ s_\psi & c_\psi & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$ being the transformation matrix relating $\langle b \rangle$ and $\langle w \rangle$ frames, only dependent of ψ, where $c_\star = \cos(\star)$ and $s_\star = \sin(\star)$. In addition, $\mathbf{R}^{\mathcal{B}}_{\mathcal{C}} = diag\left(\begin{bmatrix} 1 & -1 & -1 & 1 \end{bmatrix}\right)$. Finally, ${}^{w}\mathbf{x}_b = \begin{bmatrix} {}^{w}x_b & {}^{w}y_b & {}^{w}z_b & {}^{w}\psi_b \end{bmatrix}^\top \in \mathbb{R}^4$ represents the robot's state variables in $\langle w \rangle$, or simply $\mathbf{x}_b$.

A point on the image plane is characterized in the camera's reference frame using its intrinsic parameters obtained through calibration. The proposed representation ensures that the platform angle matches that of the image plane since the camera's frame $\langle c \rangle$ and the image plane frame $\langle i \rangle$ are parallel. By inversely transforming the camera's intrinsic parameters, the point on the image plane is obtained. This conversion yields metric coordinates measured in millimeters, necessitating division to express them in meters. Moreover, the vertical component in the image plane is zero

in its homogeneous representation due to the absence of vertical elevation. The image plane's origin is situated at the upper-left corner, denoted by $[^i u, {}^i v]^\top$. To align this origin with the principal point of the image ${}^i\mathbf{x}_d = [\mathcal{N}_u/2\ \mathcal{N}_v/2]^\top$, adjustments are necessary. Employing the pinhole camera model, assuming negligible lens distortion, the proposed transformation aims to centralize image points at the center and convert pixel values in the image plane to metric coordinates in the camera frame. This model facilitates the conversion of a binary image $\mathcal{I}$ by translating pixel positions from the image plane to metric system coordinates by

$$\begin{bmatrix} {}^c\mathbf{x}_\zeta \\ 1 \end{bmatrix} = \begin{bmatrix} {}^w z_b/\lambda \\ {}^w z_b/\lambda \\ 0 \\ \mathbf{1}_{2\times 1} \end{bmatrix}^\top \begin{bmatrix} \mathbf{I}_4 & [{}^i\mathbf{x}_c\ 0]^\top \\ \mathbf{0}_{1\times 4} & 1 \end{bmatrix} \begin{bmatrix} {}^i\bar{\boldsymbol{\zeta}} \\ 0 \\ {}^i\beta \\ 1 \end{bmatrix}, \quad (2)$$

where λ is the camera focal length and $\mathbf{I}_4$ is the identity matrix of order 4. In addition, ${}^w z_b$ is the vertical position of the drone, considering the approximation that the camera is located at 100 mm from the center of the Bebop 2. In simplified way, the model can be written as

$$\mathbf{x}_\zeta = \mathbf{x}_b + \begin{bmatrix} \alpha\, {}^w z_b({}^i\mathbf{x}_c - [\bar{\boldsymbol{\zeta}}\ 0]^\top) \times 10^{-3} \\ {}^i\beta \end{bmatrix}, \quad (3)$$

where ${}^i\mathbf{x}_c = \begin{bmatrix} {}^i\mathbf{x}_d & 0 \end{bmatrix}^\top \in \mathbb{R}^3$ is the image principal point, i.e. the center of the image plane.

Additionally, α serves as a proportionality constant converting pixel measurements to meters, incorporating the camera focal length and its displacement concerning the robot's geometric center. Through empirical measurements specific to the Bebop's camera, α was determined to be 2.22. Notably, precise determination of the platform's height ${}^w z_\zeta$ is unattainable in this approach. However, upon successful landing, the drone's height becomes known, updating the platform's height accordingly. This simplified approach stems from the analysis of successive images captured by the robot's camera. Due to perspective effects, objects appear smaller as they move away from the camera and vice versa. By correlating the real dimensions of an object in the image with the distance from the robot, we can recover the pinhole model. This model considers that the distance between the camera and the object is inversely proportional to its observed size in the image. By conducting observations and analysis, (3) derives straightforwardly, indicating that each camera has a specific suitable value for α.

An alternative method for estimating the base altitude involves determining the real length of the cross and measuring its length on the image plane in frames captured at various distances. By establishing an equation that correlates the UAV's vertical distance from the base with the observed length of the cross in pixels, the base altitude can be estimated.

The visual servoing control strategy proposed in this work is to obtain the platform's posture using computer vision and send this data as a reference for the desired posture of the aerial robot. Thus, the regulation task is given by $\mathbf{x}_d = \begin{bmatrix} x_d & y_d & z_d & \psi_d \end{bmatrix}^\top = \mathbf{x}_\zeta$. Since it is already possible to determine a desired angle value in the image plane, which is parallel to the drone's yaw axis, in three-dimensional space, the desired orientation of the robot is given by $\psi_d = {}^c\beta + \psi$. Now, all the navigation references are in the robot's workspace. This allows us to use a navigation controller in cartesian space.

Remark 1: The UAV moves following the reference and uses the docking points (platforms) as local references to avoid going backwards along the planned route.

Remark 2: The pixel velocity in the image plane corresponds to the drone's motion, resulting in a translation of this velocity into a change in the reference point in 3D space. Essentially, during the transformation, the desired point exhibits velocity, whereas in the physical world, the point remains stationary for static bases.

As discussed in [14], [16], the feature point may exhibit an associated velocity ${}^i\dot{\bar{\boldsymbol{\zeta}}}$ and ${}^i\dot{\beta}$, which is then translated into the reference motion calculated in Cartesian coordinates as $\dot{\mathbf{x}}_d$. This translation enables the enhancement of control references by converting them into trajectory tracking in the robot's operational space, thereby accelerating the robot's response. The temporal variation observed in the image plane can be described in terms of the UAV's velocities, as

$${}^i\dot{\bar{\boldsymbol{\zeta}}} = \mathbf{J}_i({}^i\bar{\boldsymbol{\zeta}}, {}^c z_\zeta)\mathbf{H}^{-1}\, {}^b\dot{\mathbf{x}}, \text{ with } \mathbf{H} = \begin{bmatrix} \mathbf{R}_{\mathcal{C}}^{\mathcal{B}} & [{}^b\mathbf{t}_c]_\times \mathbf{R}_{\mathcal{C}}^{\mathcal{B}} \\ \mathbf{0}_{3\times 3} & \mathbf{R}_{\mathcal{C}}^{\mathcal{B}} \end{bmatrix}, \quad (4)$$

where ${}^b\dot{\mathbf{x}} \in \mathbb{R}^6$ is the UAV's velocities on the robot's body, and $[{}^b\mathbf{t}_c]_\times$ represents the skew symetric matrix associated to ${}^b\mathbf{t}_c = \begin{bmatrix} {}^b x_c & {}^b y_c & {}^b z_c \end{bmatrix}^\top \in \mathbb{R}^3$. Finally, considering the feature vector ${}^i\bar{\boldsymbol{\zeta}}$, the image Jacobian matrix is given by $\mathbf{J}_i \in \mathbb{R}^{4\times 6}$. Or, simply, the image feature velocity can be calculated through numerical derivation of its position.

Remark 3: An important advantage of using this control strategy is that the motion reference is transformed into the robot's workspace, thereby avoiding the need to switch between controllers. This behavior helps prevent potential instabilities in the system.

E. The UAV Model and Motion Control Law

The mathematical model adopted for the navigation and control strategies for the FRTL follows the formulation described in [17], as illustrated in Figure 1(a). The translational coordinates of the quadrotor are denoted by $\boldsymbol{\xi} = \begin{bmatrix} x & y & z \end{bmatrix}^\top$, and its attitude is represented by the vector $\boldsymbol{\eta} = \begin{bmatrix} \phi & \theta & \psi \end{bmatrix}^\top$, which contains the roll, pitch, and yaw Tait-Bryan angles. Both ξ and η are defined with respect to the inertial reference frame $\langle w \rangle$, as depicted in Figure 1(a). The near-hover model of the UAV is expressed as

$$\ddot{\mathbf{x}} = \mathbf{R}_{\mathcal{B}}^{\mathcal{W}}\mathbf{K}_u\mathbf{u} - \mathbf{R}_{\mathcal{B}}^{\mathcal{W}}\mathbf{K}_{\dot{x}}\dot{\mathbf{x}}, \quad (5)$$

where $\ddot{\mathbf{x}} = \begin{bmatrix} \ddot{x} & \ddot{y} & \ddot{z} & \ddot{\psi} \end{bmatrix}^\top$, represents the robot's acceleration in $\langle w \rangle$, and ${}^b\mathbf{u} = \begin{bmatrix} u_{v_x} & u_{v_y} & u_{\dot{z}} & u_{\dot{\psi}} \end{bmatrix}^\top$ corresponds to the control inputs in the body frame. Similarly, ${}^b\dot{\mathbf{x}} = \begin{bmatrix} v_x & v_y & \dot{z} & \dot{\psi} \end{bmatrix}^\top$ denotes the velocity vector.

In terms of the control signals input,

$$\mathbf{u}_d = (\mathbf{R}_{\mathcal{B}}^{\mathcal{W}}\mathbf{K}_u)^{-1}(\ddot{\mathbf{x}}_r + \mathbf{R}_{\mathcal{B}}^{\mathcal{W}}\mathbf{K}_{\dot{x}}\dot{\mathbf{x}}), \tag{6}$$

where $\mathbf{K}_u$ and $\mathbf{K}_{\dot{x}}$ are diagonal matrices containing, respectively, the dynamic and drag parameters determined by an identification procedure. The vector $\mathbf{x}_r$ contains the reference control signal for a desired trajectory. Meanwhile, the vector of control signals acceptable by the Parrot drone is represented by $\mathbf{u}_d = \begin{bmatrix} u_\theta & u_\varphi & u_{\dot{z}} & u_{\dot{\psi}} \end{bmatrix}^\top \in \mathbb{R}^4$, which contemplates the u_θ and u_φ commands referring to the pitch and roll angles, while $u_{\dot{z}}$ and $u_{\dot{\psi}}$ relate to the rate of change of linear and angular velocity along the axis z^b, respectively. All commands are normalized in the interval $\mathbf{u} \in [-1, +1]$.

The reference signal $\ddot{\mathbf{x}}_r$ can be obtained using a PD feedback compensator plus a feed-forward term, which corresponds to $\ddot{\mathbf{x}}_r = \ddot{\mathbf{x}}_d + \mathbf{K}_d \tanh \dot{\tilde{\mathbf{x}}} + \mathbf{K}_p \tanh \tilde{\mathbf{x}}$. In this formulation, $\mathbf{x}_d$ is the desired task, which can be trajectory tracking or positioning. $\tilde{\mathbf{x}} = \mathbf{x}_d - \mathbf{x}$ is the control error, and $\mathbf{K}_d$ and $\mathbf{K}_p$ are positive definite gain matrices.

So, we can rewrite the control law in (6) as

$$\mathbf{u}_d = (\mathbf{R}_{\mathcal{B}}^{\mathcal{W}}\mathbf{K}_u)^{-1}(\ddot{\mathbf{x}}_d + \mathbf{K}_d \tanh \dot{\tilde{\mathbf{x}}} + \mathbf{K}_p \tanh \tilde{\mathbf{x}} + \mathbf{R}_{\mathcal{B}}^{\mathcal{W}}\mathbf{K}_{\dot{x}}\dot{\mathbf{x}}). \tag{7}$$

This controller is designed using feedback linearization technique assuming perfect knowledge of the model parameters in (7) for the Bebop 2. Under these assumptions, the proposed controllers make the system asymptotically stable in closed loop [18].

III. FRTL: Navigation and Mapping Challenge

The application of the methodology developed in this work focuses on addressing the first phase of the FRTL challenge. This phase entails mapping the environment using landing/takeoff platforms as reference points, landing on each platform once, and returning to the coastal base upon completion. The approach adopted here involves identifying bases during navigation and either mapping them before landing or landing directly on newly encountered bases. Initially, the UAV conducts an inspection and monitoring mission, following a predefined reference in an assumed obstacle-free environment. During navigation, the UAV continuously searches for new bases, determining whether they have been previously mapped and visited. Upon encountering a new base, the UAV estimates its posture and aligns itself with one of the lines of the center cross before initiating the landing routine. After landing, it stores the platform's posture and resumes its search for other bases. If a previously mapped base is encountered, the UAV continues its mission until an unmapped base is found. The mission concludes once the UAV lands on the last base and returns safely to the coastal base. The proposed solution for this challenge is detailed in Algorithm 1.

IV. Experiments, Results, and Discussion

The proposed algorithms were validated experimentally using the the Bebop 2 Parrot as the UAV. The control algorithms run in a ground station, at 30 Hz, acquiring the pose of the vehicle through an OptiTrack motion capture system configured with 17 cameras, and computing the control signals that are sent to the UAV via ROS, as presented in our previous work [15]. The setup of the FRTL arena is shown in Figure 3. In this scenario, three experiments were run to validate the proposed framework functioning for autonomous base detection and landing.

The proposed solution to the FRTL challenge was evaluated in a reduced environment representative of the competition arena, as shown in Figure 3. In this scenario, it was considered that each platform had a label, from A to D, to facilitate further analysis and discussion. In addition, the robot starts from the "Home" and must navigate with a main mission of inspection until it finds the targets, the platforms. After mapping them all, it must return to the coastal base, signaling the end of the mission.

The UAV carried out the task in around 240 seconds, landing only once on each platform, even in cases where it identified other platforms that had already been mapped and visited, ignoring them and continuing the search for a new base.

Algorithm 1 Solve the FRTL challenge.

```
Initializes an empty array to store the visited bases
Initiate inspection and monitoring mission
UAV takes off from the coastal base
while t < t_max & BatteryLevel > 10% do
    if Permission of Exectuion then
        if Found an unmapped and unvisited base then
            Estimate the posture of the base
            Update the landing routine at the base
            Start landing when appropriate
        else
            Updates the navigation reference of the main mission
        end if
        if The UAV landed at the base then
            Map the base
            Wait 5 seconds
            Start the take-off and search for new bases
        end if
        if All bases have been mapped and visited then
            Start the take-off
            Navigate to the coastal base and end the mission
        end if
    end if
end while
```

Fig. 3. The setup FRTL arena.

To illustrate some of these steps, the video `https://youtu.be/uvRnvZVthFE?t=170` and `https://youtu.be/uvRnvZVthFE?t=360` presents the UAV performing the mission, with emphasis on the color segmentation process, evaluating the flight maneuvers in the environment to demonstrate the base detection and landing strategies to solve the challenge. The UAV navigates fully autonomously from the coastal takeoff base to another bases, maps them, and performs a guided landing.

It is important to have in mind that the magnitude of the UAV's distance error from the center of the base, in the image plane, is inversely related to its altitude, as stated in Section II. After all, the higher the UAV is, the lower the resolution of the image is, consequently, the smaller the error in the horizontal plane is. In this line of reasoning, during the landing stage, when the UAV finds a base, its altitude is reduced, to later adjust its position relative to the center and to land effectively. Otherwise, if the error does not reduce, or worse, increases, the UAV returns to the previous phase, increasing its altitude for a new observation of the base and starting a new landing attempt. The results are shown in Figure 4, in which one can see the UAV's behavior in the x, y and z directions, as well as the platform orientation estimations and the yaw control performed to orientate the robot in relation with one of the cross sides.

It can be seen that the robot was able to identify four platforms and land on all of them with the orientation as close as possible to one of the vertices of the central cross, thus ensuring a smooth and safe landing. After finishing the task, the robot returned to its point of origin. To better support the discussions, Table I shows the real values $\boldsymbol{\zeta}$ and estimated values $\bar{\boldsymbol{\zeta}}$ of the pose of the platforms according to their distribution in the environment, considering the global referential located in the center of the experiment room. Additional information includes the robot's posture after landing $\boldsymbol{x}_b^{Land}$ indicating how close to the center of the platform the control system was able to take the robot during the landing execution. In this information, the first three values of each tuple are given in metric system while the last value is an angular measurement given in radians. In particular, attention is drawn to the distance of the robot in relation to the center of the platform $\|\tilde{\mathbf{x}}\|$, considered here as the control error, as well as the error in estimating the actual posture of each platform $\|\tilde{\boldsymbol{\zeta}}\|$. In these cases, it can be seen that the farthest landing from the center of the platform occurred when the robot encountered platform A, the only suspended platform. This fact can be better observed in the video provided, where it can be seen that due to the proximity of the UAV to the platform, the influence of the turbulence generated by the UAV's wings displaces the robot laterally, causing it to land further from the center of the platform. Furthermore, in general, the estimates of the posture of the platforms did not have an absolute error greater than 25 cm, which is less than a quarter of the dimension of one side of the platforms. The errors and distances were calculated using the Euclidean norm of a vector.

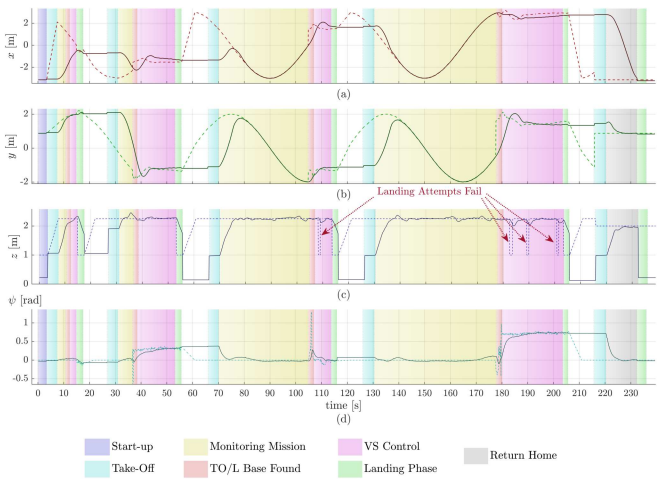

Fig. 4. The platforms' posture estimation during the mission execution.

V. Concluding Remarks

This paper introduces a servo-visual controller designed to facilitate autonomous take-off and landing for UAVs, particularly in environments where real-time interaction with various landing platforms is essential. Leveraging pattern detection and color segmentation techniques, the controller accurately identifies take-off and landing platforms while estimating their current orientation. Experimental validation across four platforms, varying in orientation, height, and position, demonstrates the system's adaptability to diverse conditions. Our work addressed the Flying Robots Trial League challenge, focusing on environment mapping using the platforms as reference dock points.

Opportunities for enhancing the proposed approach include its application to mobile platforms and the integration of Visual Simultaneous Localization and Mapping (vSLAM) for enhanced state feedback, promising further advancements in landing and mapping accuracy.

TABLE I

Estimated posture of the bases *vs.* their actual ground-thruth posture.

Parameter / Base label	**A**	**B**	**C**	**D**
$\boldsymbol{\zeta}$ ([m], [rad])	$(-0.38, 2.09, 1.06, -0.038)$	$(-1.28, -1.31, 0.15, 0.452)$	$(1.71, -1.27, 0.16, 0.139)$	$(2.74, 1.52, 0.13, -1.056)$
$\bar{\boldsymbol{\zeta}}$	$(-0.41, 2.03, 1.06, -0.096)$	$(-1.51, -1.37, 0.14, 0.355)$	$(1.52, -1.32, 0.15, -0.017)$	$(2.93, 1.57, 0.12, 0.785)$
$\boldsymbol{x}_b^{Land}$	$(-0.75, 2.02, 1.06, -0.070)$	$(-1.36, -1.19, 0.14, 0.359)$	$(1.634, -1.12, 0.15, 0.074)$	$(2.75, 1.34, 0.12, 0.720)$
$\|\tilde{\mathbf{x}}\|$ [m]	0.377	0.141	0.170	0.183
$\|\tilde{\boldsymbol{\zeta}}\|$ [m]	0.068	0.242	0.199	0.192
$\approx t_{VS+land}$ [s]	6.85	18.95	11.15	27.85

ACKNOWLEDGMENT

The authors would like to thank FAPEMIG (Fundação de Amparo à Pesquisa do Estado de Minas Gerais), process APQ-02282-24, for their support and funding, which enabled the development of this project. Prof. Dr. Brandão is also grateful for the support of a research grant from CNPq (Conselho Nacional de Desenvolvimento Científico e Tecnológico), process 309884/2022-5. M.Sc. Fagundes-Junior also thanks FAPEMIG for the scholarship that allowed him to develop his Ph.D. studies and research.

REFERENCES

[1] S. Jordan, J. Moore, S. Hovet, J. Box, J. Perry, K. Kirsche, D. Lewis, and Z. T. H. Tse, "State-of-the-art technologies for uav inspections," *IET Radar, Sonar & Navigation*, vol. 12, no. 2, pp. 151–164, 2018.

[2] Y. Zhang, X. Yuan, W. Li, and S. Chen, "Automatic power line inspection using uav images," *Remote Sensing*, vol. 9, no. 8, p. 824, 2017.

[3] S. Chen, D. F. Laefer, E. Mangina, S. I. Zolanvari, and J. Byrne, "Uav bridge inspection through evaluated 3d reconstructions," *Journal of Bridge Engineering*, vol. 24, no. 4, p. 05019001, 2019.

[4] Y. Wu, Y. Qin, Z. Wang, and L. Jia, "A uav-based visual inspection method for rail surface defects," *App. Sci.*, vol. 8, no. 7, p. 1028, 2018.

[5] Y. Tan, S. Li, H. Liu, P. Chen, and Z. Zhou, "Automatic inspection data collection of building surface based on bim and uav," *Automation in Construction*, vol. 131, p. 103881, 2021.

[6] C. O. Barcelos, L. A. Fagundes-Júnior, D. K. D. Villa, M. Sarcinelli-Filho, A. P. Silvatti, D. C. Gandolfo, and A. S. Brandão, "Robot formation performing a collaborative load transport and delivery task by using lifting electromagnets," *App. Sci.*, vol. 13, no. 2, p. 822, 2023.

[7] M. Hu, W. Liu, J. Lu, R. Fu, K. Peng, X. Ma, and J. Liu, "On the joint design of routing and scheduling for vehicle-assisted multi-uav inspection," *Future Generation Computer Systems*, vol. 94, pp. 214–223, 2019.

[8] P. M. Gupta, E. Pairet, T. Nascimento, and M. Saska, "Landing a uav in harsh winds and turbulent open waters," *IEEE Robotics and Automation Letters*, vol. 8, no. 2, pp. 744–751, 2022.

[9] A. Khazetdinov, A. Zakiev, T. Tsoy, M. Svinin, and E. Magid, "Embedded aruco: a novel approach for high precision uav landing," in *2021 International Siberian Conference on Control and Communications (SIBCON)*. IEEE, 2021, pp. 1–6.

[10] I. Lebedev, A. Erashov, and A. Shabanova, "Accurate autonomous uav landing using vision-based detection of aruco-marker," in *Int. Conf. on Interactive Collaborative Robotics*. Springer, 2020, pp. 179–188.

[11] A. Sales, P. Mira, W. Garcia, A. M. Nascimento, T. Amorim, T. Nascimento *et al.*, "Vision-based k-nearest neighbor approach for multiple search and landing with energy constraints," in *2023 Latin American Robotics Symposium (LARS)*. IEEE, 2023, pp. 71–76.

[12] C.-W. Chang, L.-Y. Lo, H. C. Cheung, Y. Feng, A.-S. Yang, C.-Y. Wen, and W. Zhou, "Proactive guidance for accurate uav landing on a dynamic platform: A visual–inertial approach," *Sensors*, vol. 22, no. 1, p. 404, 2022.

[13] M. F. S. Rabelo, A. S. Brandão, and M. Sarcinelli-Filho, "Landing a uav on static or moving platforms using a formation controller," *IEEE Systems Journal*, vol. 15, no. 1, pp. 37–45, 2020.

[14] A. Keipour, G. A. Pereira, R. Bonatti, R. Garg, P. Rastogi, G. Dubey, and S. Scherer, "Visual servoing approach to autonomous uav landing on a moving vehicle," *Sensors*, vol. 22, no. 17, p. 6549, 2022.

[15] L. A. Fagundes-Junior, C. O. Barcelos, D. C. Gandolfo, and A. S. Brandão, "Bdp-uaifly system: A platform for the robocup brazil open flying robot trial league," in *2023 International Conference on Unmanned Aircraft Systems (ICUAS)*. IEEE, 2023, pp. 1021–1028.

[16] E. Velasco-Sánchez, L. F. Recalde, B. S. Guevara, J. Varela-Aldás, F. A. Candelas, S. T. Puente, and D. C. Gandolfo, "Visual servoing nmpc applied to uavs for photovoltaic array inspection," *IEEE Robotics and Automation Letters*, 2024.

[17] A. O. Pinto, H. N. Marciano, V. P. Bacheti, M. S. M. Moreira, A. S. Brandão, and M. Sarcinelli-Filho, "High-level modeling and control of the bebop 2 micro aerial vehicle," in *2020 International Conference on Unmanned Aircraft Systems (ICUAS)*. IEEE, 2020, pp. 939–947.

[18] L. V. Santana, A. S. Brandão, and M. Sarcinelli-Filho, "Navigation and cooperative control using the ar. drone quadrotor," *Journal of Intelligent & Robotic Systems*, vol. 84, pp. 327–350, 2016.